



Bidaw: Enhancing Key-Value Caching for Interactive LLM Serving

via Bidirectional Computation-Storage Awareness



Shipeng Hu · Guangyan Zhang · Yuqi Zhou · Yaya Wei · Ziyang Zhong · Jike Chen
Tsinghua University · CUGB · China Telecom

FAST '26 Paper Seminar

발표자 : _____ · 일시 : _____

발표 순서

- 1 **Background** Interactive LLM serving 과 KV cache 의 역할
- 2 **Motivation** KV loading 병목 , 그리고 세 가지 워크로드 관찰
- 3 **Key Idea** Bidirectional computation-storage awareness
- 4 **Mechanism 1** I/O-aware request scheduling (compute side)
- 5 **Mechanism 2** Previous-answer-based eviction (storage side)
- 6 **Mechanism 3** Storage-efficient tensor caching
- 7 **Evaluation** 5 개 모델 · 50 분간 토크의 핵심 plot 7 개
- 8 **Discussion** 한계 , 일반화 가능성 , takeaway

Interactive LLM serving — 멀티 라운드 대화 패턴

사례

- Replika · Duolingo · 챗봇 — 사용자가 LLM 과 번갈아 대화
 - 사용자 평균 22.4 라운드 (P90 = 45 라운드)
 - 대화 지속 시간 평균 분 단위 , 길게는 60 분 이상

기술적 핵심

- 라운드 N 답변 = 0..N-1 라운드 KV 재사용
- 재계산 = GPU FLOPs 낭비 → caching 필수

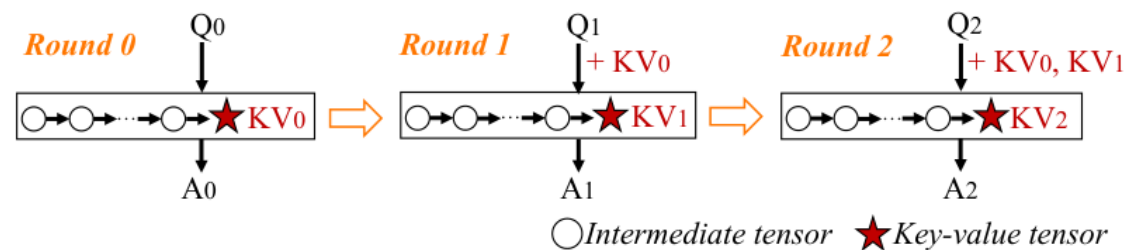


Figure 1. 각 라운드의 답변은 이전 라운드들의 KV tensor 에 의존

KV cache 의 역할 — attention 재계산을 막는 핵심 자료구조

왜 KV cache 인가

- 재계산을 안 하면 응답 latency 가 곧바로 줄어든다
- 저장 비용이 매우 큼 → 효율적 관리가 시스템 설계의 1 번 과제
- vLLM, PagedAttention 류 가속 시스템도 결국 KV cache 관리가 핵심

이번 논문의 KV 는 ?

- ▶ 한 사용자의 여러 라운드 대화 KV 를 보존 — cross-user KV 공유는 범위 밖

Self-attention(요점)

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$$

- ▶ 새 토큰 1 개의 Q 만 새로 계산
- ▶ 과거 토큰의 K, V 는 변하지 않음 → 재사용 가능

토큰 t 개 · 레이어 L · 헤드 h · head_dim d 일 때
 $\text{KV size} = 2 \cdot t \cdot L \cdot h \cdot d \cdot \text{sizeof}(\text{dtype})$

예: OPT-13B, 2,048 토큰 → 약 0.78 GB / user

왜 GPU 메모리에 다 못 두는가

- **80GB A800 GPU 1 장** → KV 가 곧바로 GPU 메모리를 점유
- **30 users/min** 도착 시 OPT-13B 동시 캐시 KV $\approx$ 480 GB
- **LLM 서버 host memory** $\approx$ GPU mem 의 1.6–3.2×
- → **host memory** 도 부족 , capacity SSD 도 사용해야 함

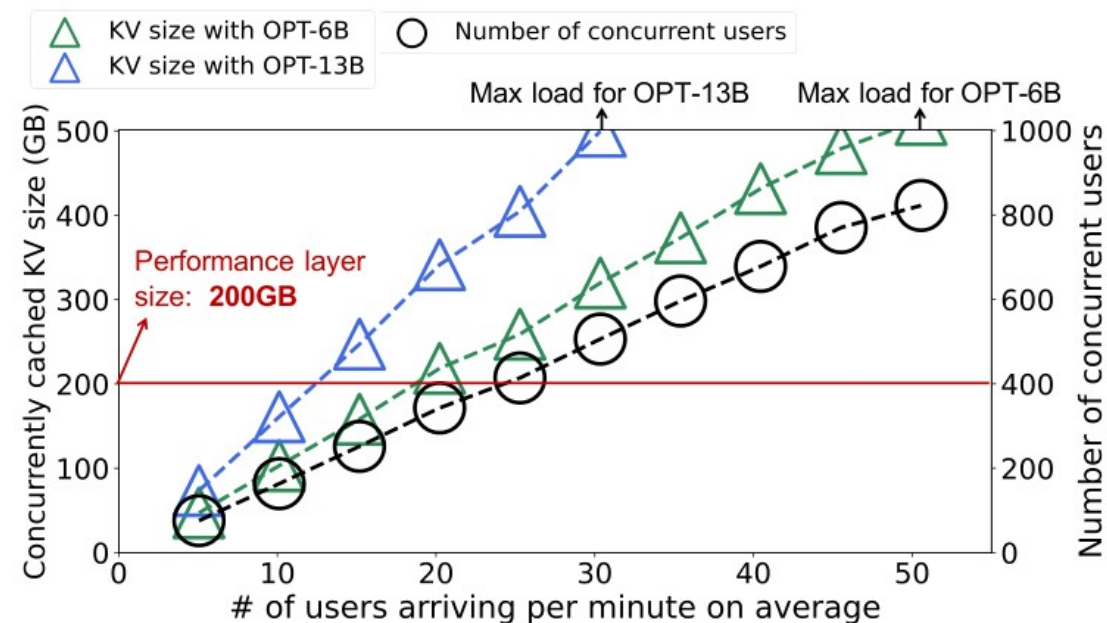


Figure 5. 사용자 도착률 vs 동시 캐시 KV 총량 (perf layer 200GB 기준)

동시 캐시 KV 는 perf layer 를 **최대 3.91×** 초과

→ 2-tier (host memory + SSD) 캐싱이 필연

기존 솔루션 — Two-tier storage caching

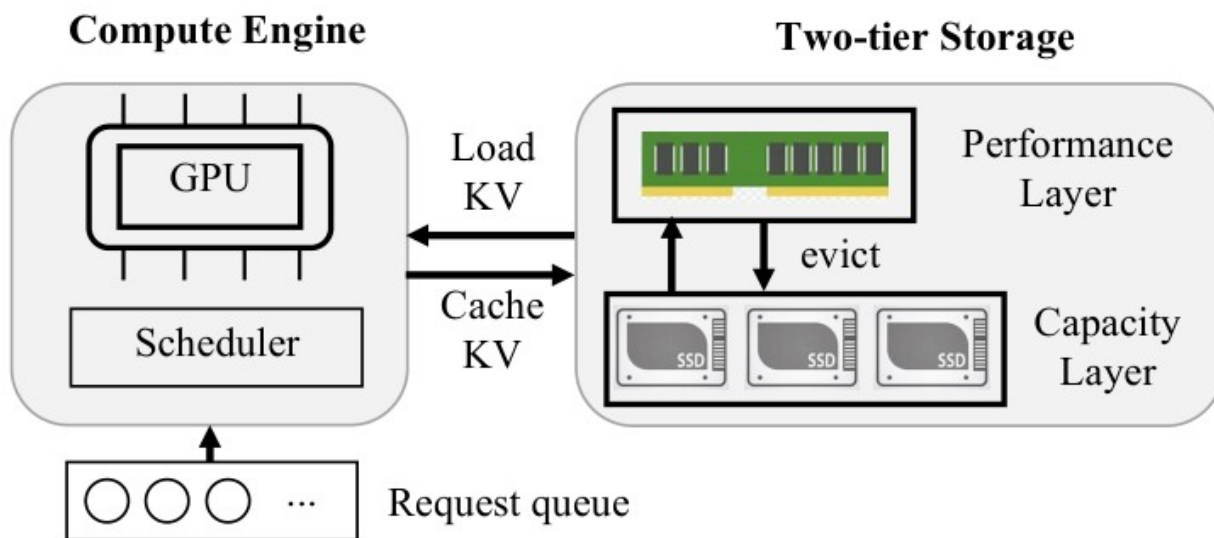


Figure 2. Compute engine ↔ two-tier storage 구조

Performance layer

Host DRAM, 빠르지만 용량 한정

Capacity layer

SSD (RAID-5, 본 논문은 1.5 GB/s)

대표 선행 시스템

- ▶ **CachedAttention** [ATC '24] queue-enhanced LRU eviction
- ▶ **FlashGen** [FAST '25] inclusive caching + GPU-fit scheduling
- ▶ **HCache** [EuroSys '25] intermediate activation 캐싱

쓰기는 백그라운드, 그러나 읽기 (load) 는 **critical path** 위에
놓여 있다

그러나 KV loading 자체가 병목 — 이상치와의 큰 격차

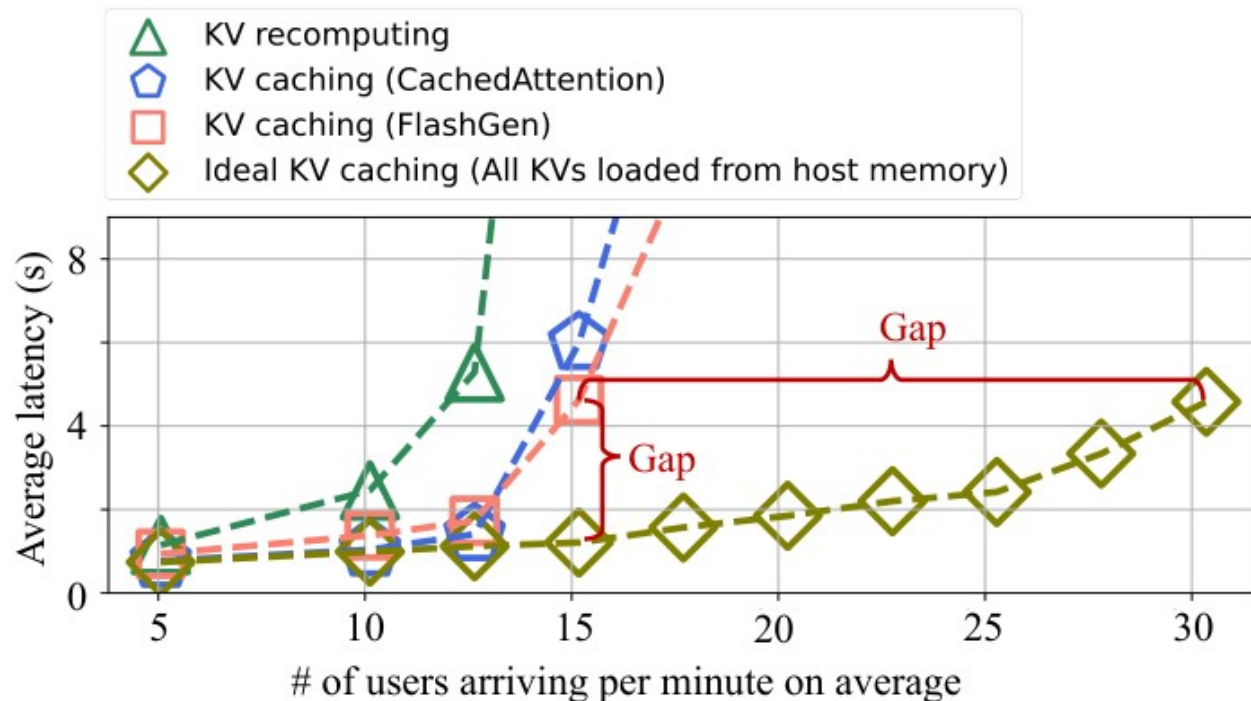


Figure 3. KV recompute / 기존 cache / Ideal cache 의 latency 비교

측정 환경

OPT-13B · A800 80GB · 200GB host · 1.5 GB/s SSD

성능 격차

- ▶ 응답 latency 가 **최대 3.8× 증가**
- ▶ throughput 은 **최대 2.0× 감소**

Ideal vs reality?

Ideal = 모든 KV 가 perf layer 에 있다고 가정한 동일 시스템 .

이 격차의 원인을 분석하는 것이 본 논문의 출발점이다.

워크로드 — 백만 라운드 실서비스 인터랙티브 대화 trace

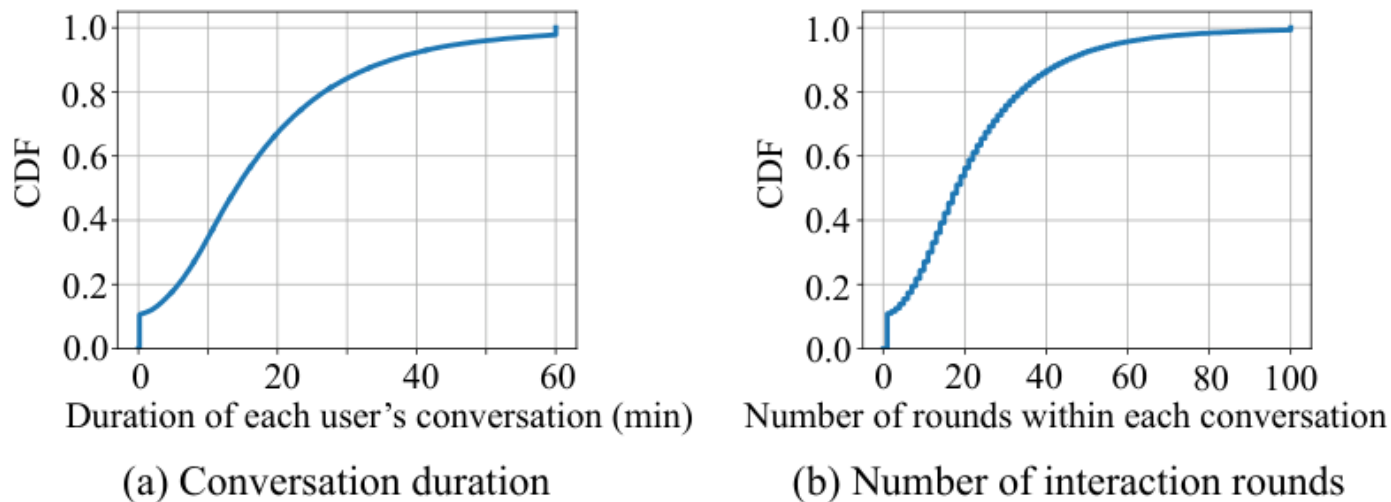


Figure 4. (a) 사용자별 대화 지속시간, (b) 라운드 수 CDF

Trace 출처

China Telecom · 1M+ 라운드 · user timestamp 보존

ShareGPT vs Mooncake와의 차이

- ▶ ShareGPT: 5.7 라운드 — interactive 분석엔 짧음
- ▶ Mooncake: 12k+ 토큰 — 길지만 대화 패턴 부재
- ▶ 본 trace: 22.4 라운드 / 36 토큰 — interactive 부합

이어지는 세 슬라이드 → **Observation 1 · 2 · 3**

Observation 1 — 동시 캐시 KV 양이 perf layer 를 크게 초과

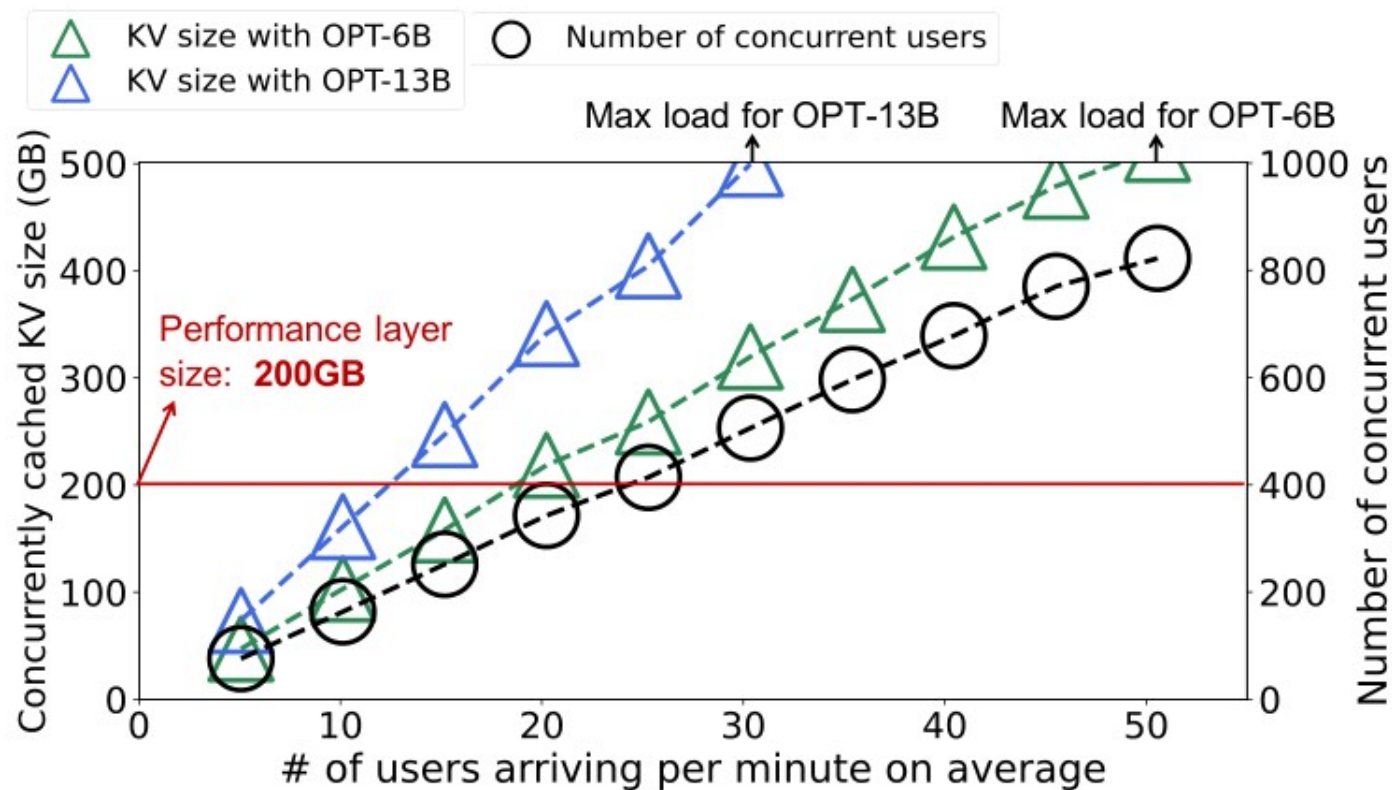


Figure 5. 사용자 도착률 vs 동시 캐시 KV 총량 / 동시 사용자 수

관찰

- ▶ 22.4 라운드 동안 KV 생존
- ▶ 도착률 ↑ → 동시 사용자 수 선형 증가
- ▶ 30 users/min $\approx$ 480 GB > perf 200 GB

함의

- ▶ perf layer 가 커도 결국 한계
- ▶ eviction 부실 → capacity layer 로 빈번히 fallback

Observation 2 — KV access 의 temporal locality 가 매우 약함

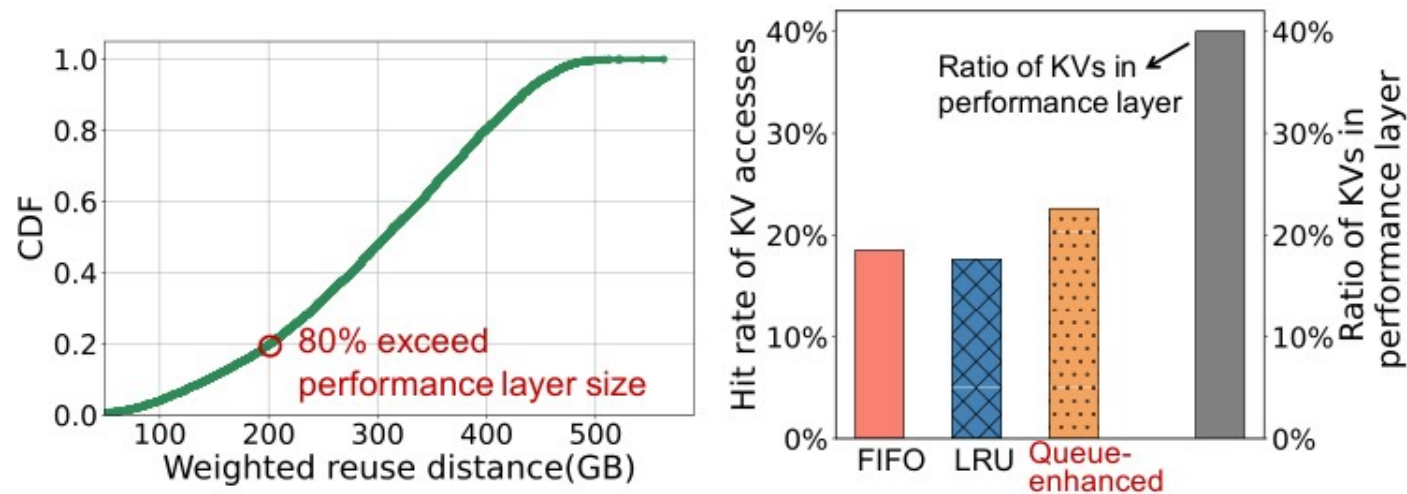


Figure 6. (a) Weighted reuse distance CDF, (b) hit rate

Weighted reuse distance

두 access 사이에 접근된 다른 KV 의 총 크기 합

수치

- ▶ 80% access > perf 200 GB
- ▶ 정책 무관 hit $\approx 20\%$
- ▶ perf 가 40 % 담아도 hit 절반 수준
→ user 가 답 읽는 사이 다른 KV 끼어들

Observation 3 — KV loading 시간 분산이 매우 큼

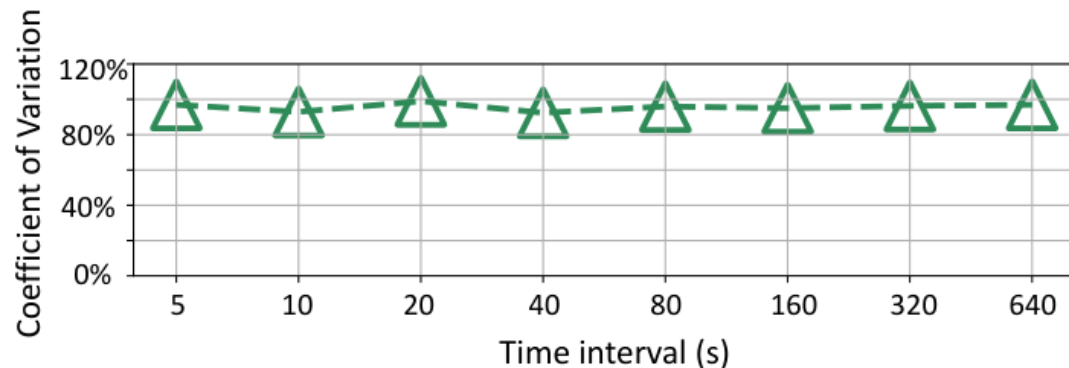


Figure 7. 시간 구간별 KV loading time 의 변동계수 CV

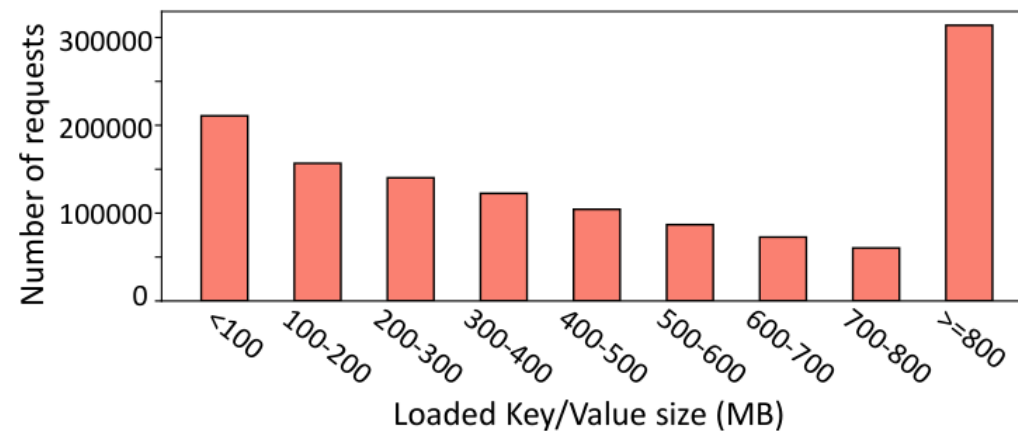


Figure 8. Loaded KV 크기 분포 (히스토그램)

왜 분산이 큰가

▶ host DRAM ↔ SSD bandwidth 격차 · 요청별 KV size 가 수십 MB ~ 수백 MB · 5 초 윈도우에서도 CV > 90 %

함의 — 큰 KV 한 개가 뒤따르는 작은 KV 까지 GPU idle 로 만든다

Root cause — compute engine 과 storage 가 서로 unaware

문제 1. Compute engine 은 storage I/O latency 를 모른다

- ▶ 어떤 요청의 KV 가 perf layer 에 있는지 , capacity layer 에 있는지
 - ▶ 어떤 요청의 KV size 가 큰지 vs 작은지
- 그냥 FCFS 로 dispatch → 큰 I/O 가 큐 머리에 박히면 GPU idle

문제 2. Storage 는 사용자 대화 패턴을 모른다

- ▶ Eviction 은 자기 측의 KV access history 만 사용
 - ▶ 사용자의 다음 질문 timing 은 compute 가 만든 답변 길이에 좌우
- 정보 부재로 인해 hit rate $\approx$ 20% 수준

문제는 둘 다 정보가 한쪽 방향으로만 흐르거나 , 아예 흐르지 않는다는 것

Bidaw 의 가설 : 두 방향 모두 정보를 흐르게 하면 두 문제 모두 해결된다

핵심 아이디어 — Bidirectional awareness

Compute → Storage

이전 라운드의 **model answer 길이**를 storage 에 넘긴다

- ▶ 이 길이가 다음 질문 도착 시점의 예측 신호
- ▶ Storage 는 신호로 next-access reuse distance 추정
- ▶ hit potential 최저 KV 를 evict

Storage → Compute

각 요청 KV 의 **위치 (layer) 와 크기**를 compute 에 넘긴다

- ▶ Compute 는 KV 위치 / 크기 알고 dispatch
- ▶ ready / preparing 두 큐로 I/O 길이를 분리
- ▶ GPU idle 제거 + 큐잉 지연 감소

시스템 개관 — 3 개 컴포넌트 + 양방향 신호

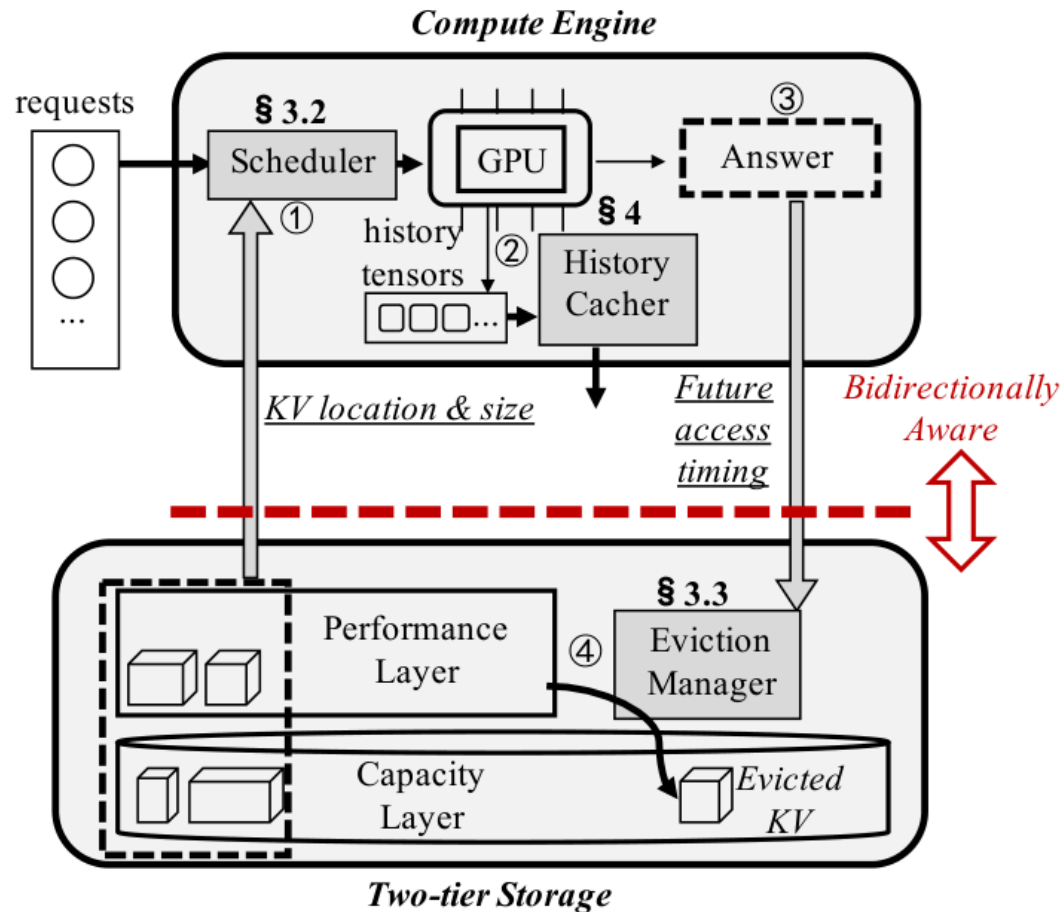


Figure 9. Bidaw 시스템 구조

① Scheduler (§3.2)

Storage로부터 KV 위치와 크기를 받아 ready/preparing 큐로 분리

② History Cacher (§4)

GPU에서 생성된 history tensor 중 storage-efficient tensor만 캐시

③ Eviction Manager (§3.3)

Compute가 넘긴 답변 길이로 reuse distance 추정 → hit potential 최저 KV 축출

Bidirectional 신호

- ▶ Storage → Compute: KV location & size
- ▶ Compute → Storage: future access timing (=answer length)

End-to-end workflow

- ① 사용자 요청 도착 → Scheduler 가 KV 의 location/size 를 storage 에서 조회
→ Ready queue 또는 Preparing queue 로 dispatch
- ② GPU 가 inference 진행 중 생성한 storage-efficient tensor 를 History Cacher 가 캐싱
→ KV tensor 대신 더 작은 intermediate tensor 를 저장 (MHA-only)
- ③ GPU 응답 완료 → 답변 토큰 길이를 Eviction Manager 에게 전달
→ 다음 KV access 의 reuse distance 분포 추정
- ④ Free perf-layer 공간이 임계 미만이면 eviction 트리거
→ Hit potential 최저 KV 를 capacity layer 로 이동 (inclusive caching)

Mechanism 1 동기 — I/O blocking 과 overlap 의 한계



Figure 10. I/O-aware request scheduling 전략 개관

왜 단순 overlap 이 안 되는가

- ▶ 한 iteration 의 GPU compute = 수십 ms
- ▶ Capacity layer I/O = 수백 ms
- ▶ 다음 layer 의 KV load 와 overlap 해도 시간 차가 너무 큼
- ▶ naive overlap 은 “다음 토큰 1 개를 기다리며 GPU 가 또 idle”

설계 목표 — 느린 I/O 요청이 빠른 I/O 요청을 막지 않게 하면서, **큰 KV 요청도 starvation 없이 진행시킨다**

I/O-aware scheduling — Dual queue + disk-HRRN

Dual queue

Ready queue

KV 가 perf layer 에 있는 요청 — FCFS, GPU 에 즉시 dispatch

Preparing queue

KV 가 capacity layer 에 있는 요청 — 백그라운드로 perf layer 로 끌어올림

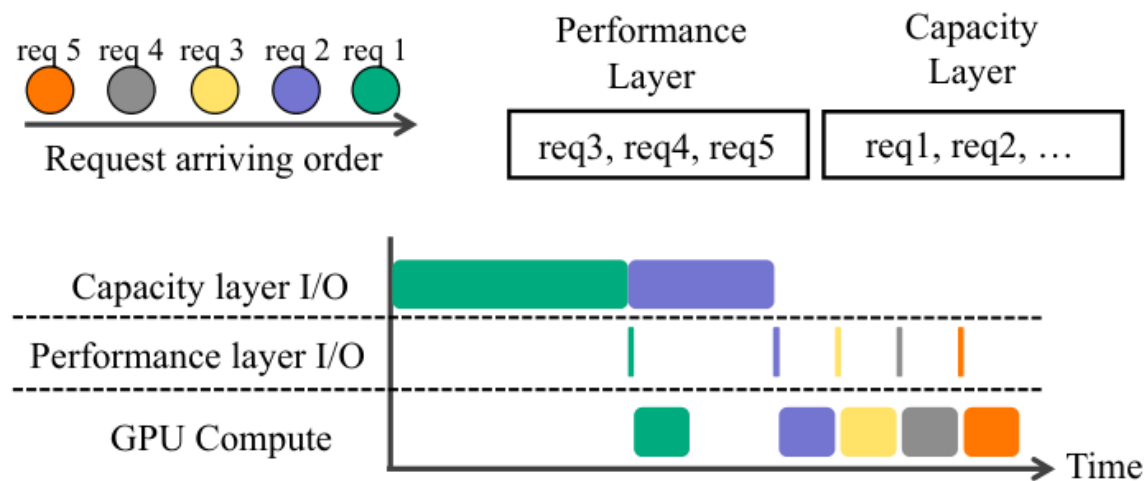
promotion 시 **원래 도착 시각을 유지**해 tail latency 폭주를 방지

disk-HRRN (preparing queue 정렬)

Response ratio = $1 + (\text{Request waiting time}) / (\text{KV size})$

- ▶ 작은 KV 먼저 → **ready queue** 로 빨리 promote
- ▶ waiting ↑ 시 큰 KV 도 promote (기아 방지)
- ▶ classical HRRN 의 I/O time $\approx$ KV size 변형

동작 예시 — GPU idle 을 어떻게 없애는가



(a) I/O-oblivious request scheduling (FCFS)

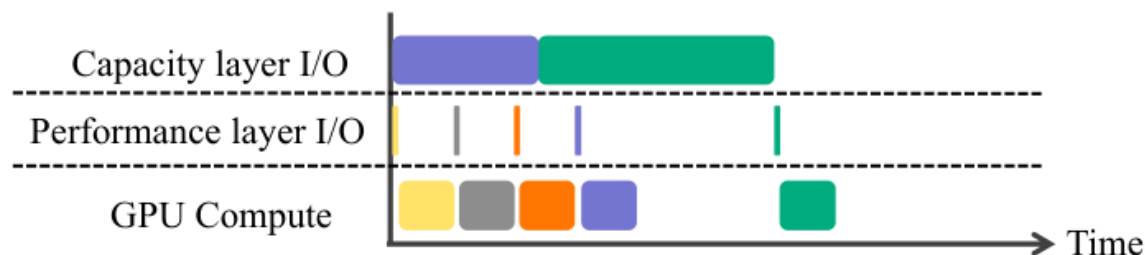


Figure 11. (a) FCFS vs (b) I/O-aware scheduling 의 timeline 비교

(a) FCFS

req 1, 2 (큰 capacity-layer KV) 가 머리에서 대기 →

GPU 와 perf-layer I/O 가 모두 idle

(b) I/O-aware

req 3, 4, 5 (perf-layer hit) 먼저 GPU 로 → 그 사이 req 2 가 promote

큐잉 시간 평균 **5.76s → 2.45s (-57.5%)**

Mechanism 2 핵심 관찰 — 답변 길이 $\leftrightarrow$ 다음 reuse distance

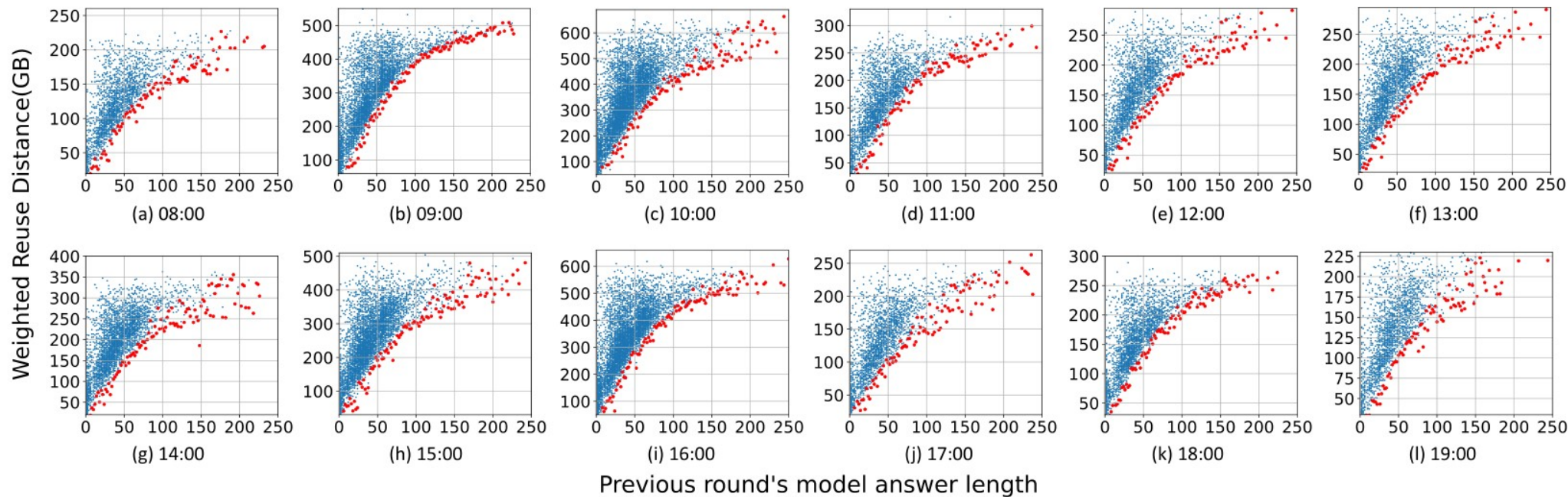


Figure 12. Hour 별로 답변 길이(가로축) vs 다음 KV access 의 reuse distance(세로축). 빨간 라인 = 하한

정의 — Weighted reuse distance

현재 access $\leftrightarrow$ 다음 access 사이에 접근되는 다른 KV 의 총 byte 합

관찰 — 답변이 길수록 사용자가 다음 질문을 늦게 보낸다 $\rightarrow$ 그 사이 다른 사용자 KV 가 끼어들어 reuse distance 하한이 **단조 증가 (Spearman $\rho = 0.94 \sim 0.98$)**

그러나 큰 reuse distance 라고 무조건 미스가 아니다

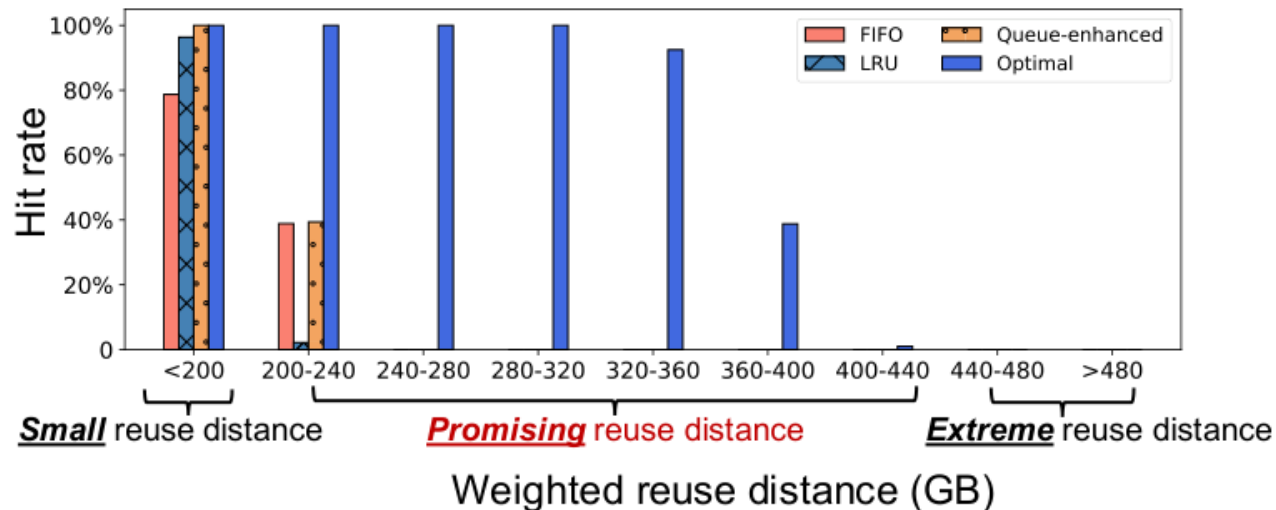


Figure 13. Reuse distance 구간별 hit rate (Optimal vs FIFO/LRU/Q-enh)

세 영역으로 분할

Small (< perf layer size)

어떤 정책이든 hit ≈ 1.0

Promising (중간 영역)

Optimal(Belady) 은 100% 가까이 유지하지만 LRU 는 0~40% 수준

Extreme (440 GB+)

Optimal 도 hit ≈ 0 — eviction 후보 1 순위

교훈 — distance 만 보고 evict 하면 promising 구간을 잃는다.
Optimal 과의 격차를 좁히려면 **구간별 hit 확률을 모델링**해야 한다.

Eviction 알고리즘 — Ghost cache + bucket

- ① Promising 영역을 m 개 fine-grained bucket 으로 분할
- ② 각 bucket 의 hit rate 를 ghost cache 위에서 Belady 시뮬레이션으로 측정
- ③ 사용자별 과거 KV access 분포로 다음 access 가 각 bucket 에 떨어질 확률을 추정
- ④ Compute 가 넘긴 답변 길이로 reuse distance 하한을 결정 → 더 작은 bucket 의 확률을 0 으로 truncate
- ⑤ Equation 2 로 overall hit potential 계산 → 가장 낮은 KV evict

Eviction 결정식 — Overall hit potential

$$\text{Overall_potential} = \text{prob_small} \cdot 1.0 + \text{prob_extreme} \cdot 0.0 + \sum_i \text{prob_promising}(i) \cdot \text{hit_promising}(i)$$

Equation (2). 가장 낮은 hit potential 을 가진 KV 를 evict

- ▶ **prob_small** 다음 access 가 perf layer 안쪽으로 들어옴 — hit 확률 1.0
- ▶ **prob_promising(i)** promising bucket i 에 떨어질 확률 (m 개)
- ▶ **hit_promising(i)** ghost cache 의 Belady 시뮬레이션이 알려주는 i bucket 의 hit rate
- ▶ **prob_extreme** perf layer 로 영원히 안 돌아옴 — hit 확률 0

Note. Ghost cache 는 백그라운드에서 Belady 시뮬레이션을 수행 — eviction 결정 단계에는 **0.35 ms** 만 추가 (자세한 비교는 평가 슬라이드에서)

Mechanism 3 — 어떤 tensor 를 캐시할 것인가

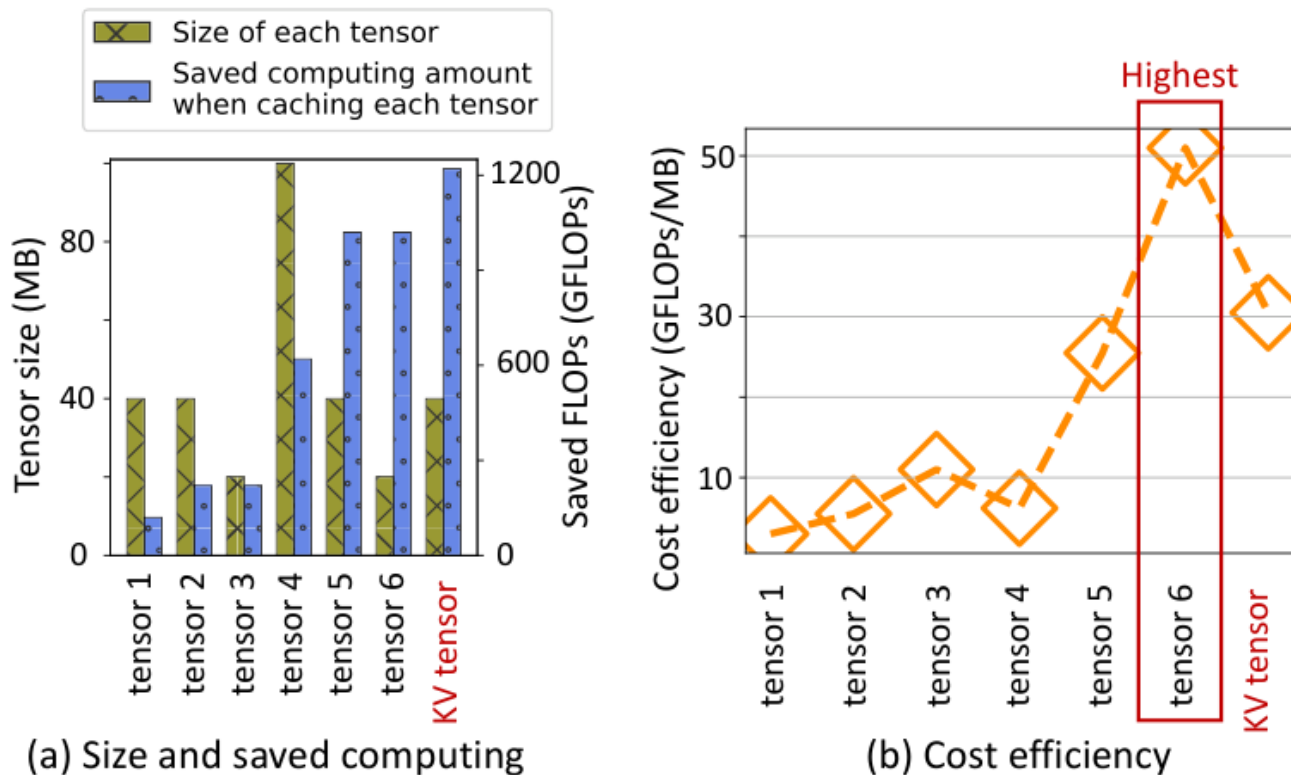


Figure 14. (a) Tensor 별 size & 절약 가능한 GFLOPs, (b) cost efficiency

왜 KV tensor 가 최적이지 아닌가

▶ KV tensor 직접 필요 — 그러나 size 가 큼

Cost Efficiency

= Saved compute / Required space (GFLOPs/MB)

Tensor 6 (정규화된 activation)

▶ 51 GFLOPs/MB — KV tensor(30.5) 의 1.67×

▶ KV tensor 로 변환은 **GPU 1 step** 이면 충분

Storage-efficient tensor caching — 적용 범위

MHA-based 모델

- ▶ Llama · Qwen · Bloom · OPT · Baichuan
- ▶ head 별 K, V 분리 → KV 가 큼
- ▶ Tensor 6 캐싱 = 적은 공간 / 많은 compute 절약
→ Bidaw에서는 **tensor 6** 를 캐시

GQA-based 모델

- ▶ 여러 query head 가 K, V 를 공유 → KV tensor 자체가 작음
- ▶ tensor 6 의 cost-efficiency 우위가 사라짐
→ GQA에서는 그냥 **KV tensor** 를 캐시

구현 노트 — vLLM 위에 얹는 세 가지 트릭

Continuous batching

조기 종료된 요청을 배치 안에서 즉시 release → 새 요청을 빠르게 admit (ORCA 류와 호환)

Mix-grained PagedAttention

history/query 토큰은 256-token 큰 블록, response 토큰은 16-token 작은 블록 → CPU ↔ GPU 대역폭 활용 + 단편화 감소

Low-priority CUDA stream

Storage-efficient tensor → KV tensor 변환을 별도 스트림에서 수행 → idle SM 활용, 본 추론 latency에 영향 없음

Inclusive caching

Eviction 시 capacity layer에 이미 사본이 있어 큰 write 트래픽이 발생하지 않음 (FlashGen 류와 동일)

실험 환경 및 비교 대상

Hardware / Software

GPU: NVIDIA A800 80GB

Host mem: 200 GB (perf layer)

Storage: RAID-5 over 4× SATA SSD, 1.5 GB/s

PCIe: Gen 4 (~30 GB/s)

Models: OPT-6.7B/13B/30B, Qwen-7B/14B

Workloads: 자체 1M-round trace, ShareGPT
(Poisson 시뮬)

Baselines

- ▶ **vLLM** Recompute (no KV caching)
- ▶ **CachedAttention** Queue-enhanced LRU + 2-tier
- ▶ **FlashGen** Inclusive caching + GPU-fit scheduling
- ▶ **Optimal** All KVs in perf layer (upper bound)
- ▶ **Bidaw** 본 논문

Overall performance — 5 개 모델 동시 평가

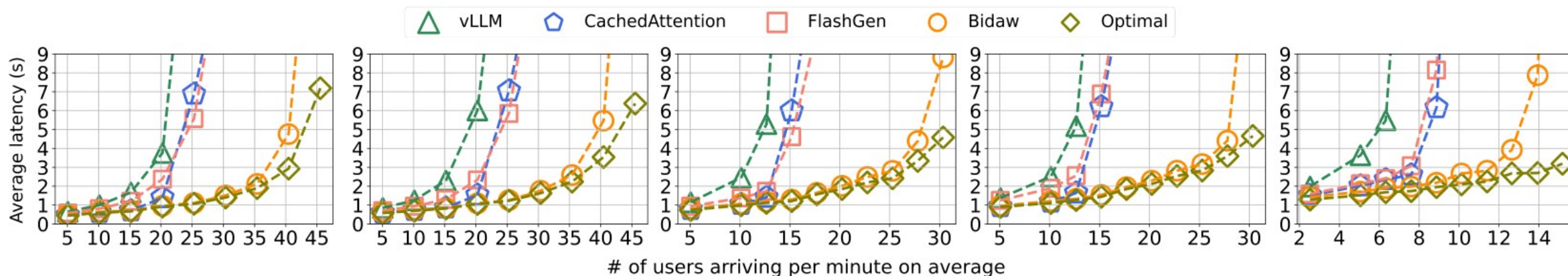


Figure 15. 사용자 도착률 vs 평균 응답 latency, 5 개 모델

- ▶ **Bidaw** — 응답 latency 최대 3.58× 감소 (vs CachedAttention/FlashGen)
- ▶ **Bidaw** — throughput 1.43–1.83× 향상 (동일 latency 기준)
- ▶ **Optimal** 과의 gap 거의 없음 — perf-layer-only 수준에 근접
- ▶ **Note** Bidaw 는 lossless — 답변 정확도는 FlashGen 등과 동일

Memory sensitivity & Eviction miss rate

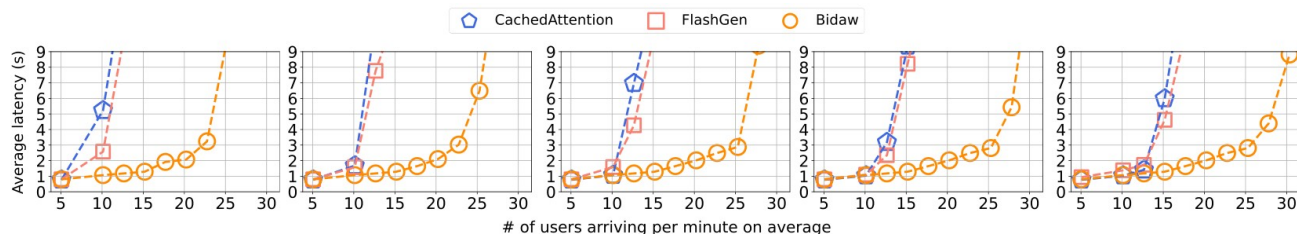


Figure 16. host memory 120~200GB 범위에서의 latency

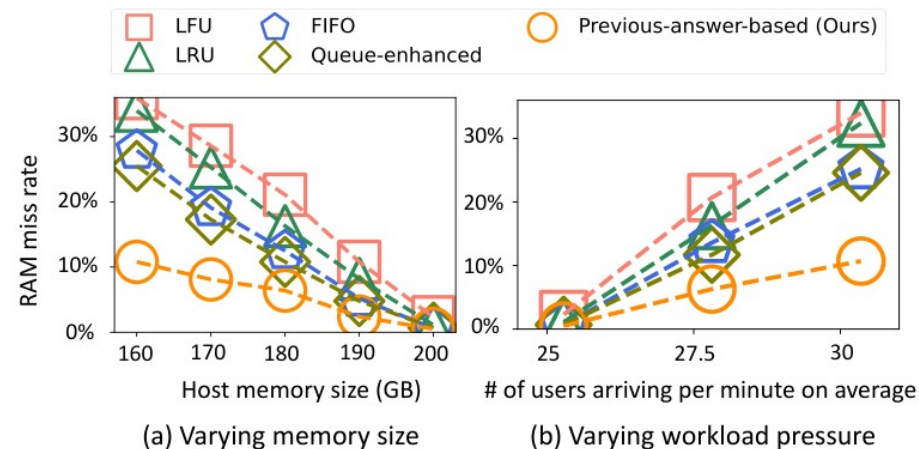


Figure 18. (a) memory size · (b) workload pressure

Memory sensitivity (Fig. 16)

- ▶ host memory 120GB 까지 줄여도 Bidaw 는 baseline 200GB 수준의 latency
- ▶ 1.75~2.19× 더 많은 사용자 / 분 지원 — 작은 메모리에서 격차가 더 벌어짐

Eviction miss rate (Fig. 18)

- ▶ queue-enhanced (CachedAttention) 대비 **-57.6% miss rate**
- ▶ LFU/LRU/FIFO 일반 정책 대비 **-69.9% miss rate**

Tail latency · Ablation · Overhead

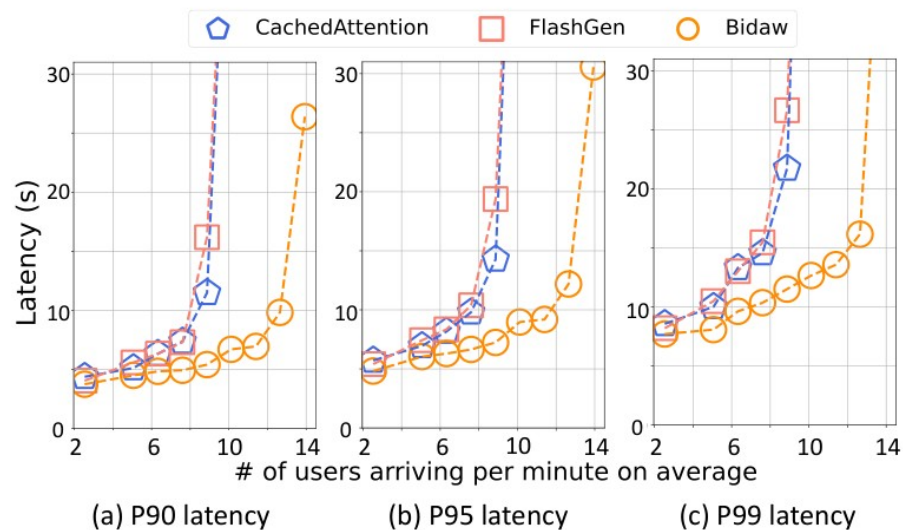


Figure 22. Tail latency P90 / P95 / P99 (OPT-30B)

- **Tail latency** P90 –52.96% / P95 –49.30% / P99 –47.03% (vs CachedAttention)
- **Ablation** Scheduling +1.58×, Eviction +1.25×, Tensor caching +1.10×
- **Overhead** Scheduling 0.62 ms, Eviction 0.35 ms, KV transform <0.1 s

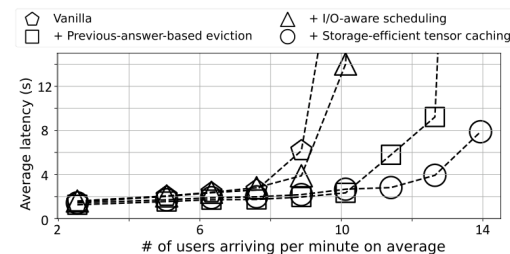


Figure 21. Ablation

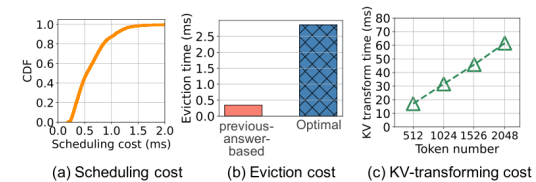


Figure 20. Overhead

정리 · 한계 · 시사점

기여

- ▶ compute ↔ storage 양방향 정보 공유라는 새로운 설계 원칙 제시
- ▶ I/O-aware scheduling + previous-answer-based eviction 의 두 메커니즘
- ▶ MHA 에서 storage-efficient tensor 캐싱으로 공간 / 연산 trade-off
- ▶ 5 모델 평균 3.58× latency / 1.83× throughput, optimal 근접

한계

- ▶ 단일 사용자 대화 안에서의 KV 재사용만 다룸 — cross-user 공유는 범위 밖
- ▶ GQA 에서는 storage-efficient tensor 효과가 사라짐
- ▶ ShareGPT 류 timestamp 부재 trace 에선 eviction 효과가 약화
- ▶ 단일 GPU 노드 평가 — disaggregated/ 멀티노드는 future work

논의

- ▶ “대화 시스템” 의 사람 - 속도가 storage 정책에 신호로 활용 가능하다
- ▶ 고전 storage 기법 (HRRN, Belady ghost cache) 이 LLM 에서 다시 빛난다
- ▶ ML serving + storage co-design 의 좋은 사례 — 후속 연구가 활발할 영역